



COLORADO SCHOOL OF MINES.
EARTH • ENERGY • ENVIRONMENT

CSCI 370 Final Report

The Prompt Pioneers

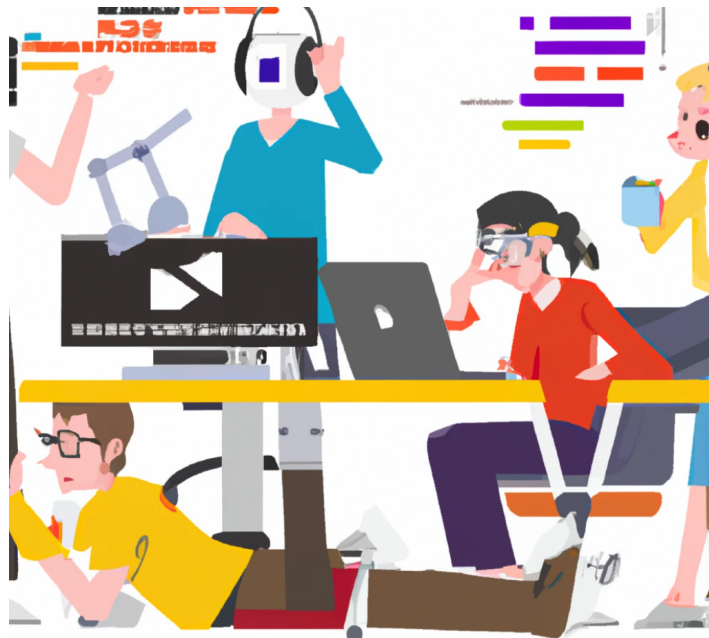
Jason Lee

Aaron Matthew Jones II

Yonghao Li

Andrew Vargish

Revised June 04, 2023



CSCI 370 Summer 2023

Dr. Zibo Wang

Table 1: Revision history

Revision	Date	Comments
New	May 19th	Completed Sections: - Cover Page - Revision History I. Introduction II. Functional Requirements III. Non-functional Requirements IV. Risks V. Definition of Done XII. Team Profile - References - Appendix A – Key Terms
Rev – 2	May 27th	Review and Rewrote (if needed) - Cover Page - Revision History I. Introduction II. Functional Requirements III. Non-functional Requirements IV. Risks V. Definition of Done XII. Team Profile - References - Appendix A – Key Terms Completed Sections: VII. System Architecture
Rev – 3	June 3rd	Review and Rewrote (if needed) - Cover Page - Revision History I. Introduction II. Functional Requirements III. Non-functional Requirements IV. Risks V. Definition of Done XII. Team Profile - References - Appendix A – Key Terms VII. System Architecture Completed Sections:

		VII. Software Test and Quality (Not including test results) VIII. Project Ethical Considerations
Rev - 4	June 11th	Review and Rewrote (if needed) Cover Page Revision History I. Introduction II. Functional Requirements III. Non-functional Requirements IV. Risks V. Definition of Done XII. Team Profile References Appendix A – Key Terms VII. System Architecture VII. Software Test and Quality (Including test results) VIII. Project Ethical Considerations Started Sections (Premature because this felt helpful at this stage): IX. Results X. Future Work XI. Lessons Learned Acknowledgements
Rev-5	June 14	Review and Rewrote(if needed) Cover Page Revision History I. Introduction II. Functional Requirements III. Non-functional Requirements IV. Risks V. Definition of Done XII. Team Profile References Appendix A – Key Terms VII. System Architecture VII. Software Test and Quality (Including test results) VIII. Project Ethical Considerations Finished. IX. Results X. Future Work XI. Lessons Learned

Table of Contents

Table of Contents..... 3

I. Introduction..... 4

II. Functional Requirements..... 5

III. Non-Functional Requirements..... 6

IV. Risks..... 7

V. Definition of Done..... 8

VI. System Architecture..... 9

VII. Technical Design..... 11

VIII. Software Test and Quality..... 14

IX. Project Ethical Considerations..... 17

X. Results..... 18

XI. Future Work..... 19

XII. Lessons Learned..... 20

XIII. Acknowledgments..... 20

XIV. Team Profile..... 21

References..... 22

Appendix A – Key Terms..... 23

I. Introduction

Purpose/Goal of The Project

The purpose is to explore better how to use self-promoting systems and agents to make a new task bot that answers client questions similar to ChatGPT but with the ability to have agents that reference client data and useful internet data as fits to answer the required first question. The chatbot will create a report summary based on the end result of completing a self-made task list and then using agents to accomplish each task.

More specifically, we will focus on three main types of questions the business client may ask about and we will focus our babyAGI, an open-source version of the aforementioned system, to create an optimized summary from “agents”: the documents vector database (a type of database that stores vectors that is compatible with ChatGPT), relevant internet resources, and the knowledge of GPT.

Client

James Taylor is a former Boulder graduate who then went to MIT for business. His work in recent years has been related to data processing. More recently, he has been a technical advisor for multiple data companies. Additionally, he helped form Analytical Data Systems (ADS) which offers “consulting services to maximize your startup's potential and set it up for success” (James Taylor’s Indeed page). Before this, he had demonstrated excellence in these many categories as a former CTO/Senior Software Engineer of Symbiotic Engineering, ICF, TSYS, Compuware, etc...

Client’s Goals

James Taylor’s current focus is on consulting with other businesses about the use cases of new Artificial Intelligence (AI) technologies. Currently, these companies that Mr. Taylor advises have garnered interest in GPT technology and he wants a team of Mines students to effectively do research and produce deliverables that he can show to clients as possible use cases.

Existing Software Platform

There was some starting code that our client gave us that provided a User Interface (UI) with Pinecone that used Typescript, and LangChain(see appendix). We used this as one of our main bases to build our project. Additionally, the second biggest resource that we referenced is a GitHub repository containing BabyAGI using the BabyCatAGI version that was posted on May 12th, 2023.

[BabyAGI](#)

[AutoGPT-Typescript](#)

[LangChain-Chat](#)

Source of Data

In order to test the application by including company information, we will use a 10K report from Tesla as example data to test on as well as other online PDFs. These documents will include [10-K forms](#) and PDFs that are publicly available online for training data. In earlier forms of the software, we used a PDF of the [constitution](#) to ensure the architecture is properly accepting documents as input.

Hardware Interface

The user chats with the AI by typing questions using the keyboard and then the display will show the chat page including the questions and answers.

Definitions, Acronyms and Abbreviations

- **AGI**: AGI stands for Artificial General Intelligence. This term refers to a type of artificial intelligence that has the capacity to understand, learn, and apply its knowledge to a broad range of tasks at a level equal to or beyond human capability. Unlike narrow AI, which is designed to perform a specific task, such as image recognition or language translation, AGI can theoretically perform any intellectual task that a human being can do.
- **LangChain**: A software framework created to simplify the development of applications that use LLMs. As a language model integration framework, LangChain is used in similar ways to language models in general, including tasks such as document analysis, summarization, chatbot creation, and code analysis.
- **10-K**: A 10-K report is an annual report required by the U.S. Securities and Exchange Commission (SEC) that gives a comprehensive summary of a company's financial performance.
- **GPT**: GPT stands for "Generative Pre-trained Transformer". It's a type of artificial intelligence model developed by OpenAI for natural language processing tasks. GPT is designed to generate human-like text by predicting the probability of a word given the previous words used in the text. It's pretrained on a large corpus of text from the internet and then fine-tuned for specific tasks, like translation, summarization, or question answering.
- **LLM**: LLM stands for Large Language Model. It refers to machine learning models that are trained on a vast amount of text data and are capable of generating human-like text. These models, like GPT-4 developed by OpenAI, are designed to understand context, make predictions, and produce responses based on the information they have been trained on. They're used in a variety of applications, including text generation, translation, summarization, and more.
- **Pinecone**: A vector database that facilitates the creation of high-performance vector search applications. It is designed to be developer-friendly, fully managed, and easily scalable without the need for managing infrastructure.

Stakeholders/Users of the Project

Anyone who wants to use AI chat to find answers from specific documents. This will mostly include the clients of ADS.

II. Functional Requirements

- Interface (console or UI)
- Makes 8-12 goals based on API
- Determines if those goals are internet (GoogleSearchAPI), base agent (default ChatGPT), or vector task(LangChain from documents stored in Pinecone)
- Finishes each goal with a summary
- Combines all into a final report with a report for the summary as well.

Main Optimization Parts

Below are the main optimization parts we identified.

We split this into two main parts of focus: Fine-tuning the model for one search and then controlling the model so that it makes more tasks, it doesn't lose focus.

Fine Tuning:

- Preprocess the model: There is one document that is already innate in the program under the data folder. This document is a PDF by the US small business association that tells how to make a business plan. This knowledge will be useful for any application of this business bot so we made sure it was trained with this data beforehand. It has been "preprocessed" by taking out citations and images to make it easier to read and not make the GPT model get off task if it uses it as an agent.
- Self-Referencing: There is a class called redoGPT that is used multiple times throughout this program to make sure that the GPT response is true. It takes in a string as a parameter such as "2+2=5" and then re-prompts GPT to rewrite it again to make sure so then it returns "2+2=4".
- We have a document called surveyed prompts where we took a snippet from our output showing the task list and asked people if it was accurate and one sentence distilled to help it in its future business tasks. We then put these distilled new prompts back into GPT to make a more accurate task list

Control:

- We have the redoGPT check at every task creation step to make sure that the specific task hasn't gone off task. This makes sure that as the tasks go deeper and deeper in, there is less hallucination.
- We used GPT-4 and low temperature as the starting model and parameter so that the very first task creation list is minimally random and so that it can start off on strong footing which significantly reduced the tasks going off on a tangent.

III. Non-Functional Requirements

The above functional requirements are supported by non-functional requirements (also known as quality requirements), which impose constraints on design or implementation

Examples include performance requirements, cost constraints, security, and reliability.

Performance Requirements:

We needed multiple API keys (for Google search, chat GPT, and Pinecone) that were used for the LangChain and BabyAGI implementation, which all had cost constraints associated.

Security:

There will be a possible security risk pertaining to who has access to the confidential data that the companies enter as it must be sent through ChatGPT so that it can be prompted and stay consistent with the required documentation.

Reliability:

There were reliability risks as this process required the Agents to self-prompt themselves, which meant that there was potential for these systems to go off in directions that may not be aligned with the prompt that spawned them. This was a constant issue that needed to change depending on the questions, the LLMs, as well as the prompting code.

Usability:

This will be tailored for different clients but generally, it's supposed to focus on using documents that they provide. We had to try different implementation techniques to see which software solutions were the most aligned with the client's wants and needs.

IV. Risks

Here are some of the risks that we encountered during our project. Included with their description is their likelihood, impact, and possible risk mitigation.

Risk: "Hallucination" of GPT Model causing unsupported and possibly false business facts.

Likelihood: Very Likely

Impact: Moderate

Risk Mitigation:

1. Give fact-checked links.
2. Implement a feedback mechanism to allow the user to clarify or correct any misinterpretations.
3. Incorporate context awareness into the system to improve task understanding and accuracy.

Risk: Lack of control and accountability

Likelihood: Likely

Impact: Moderate

Risk Mitigation:

1. Implement a user interface that allows users to review and approve individual steps before execution.
2. Provide an option to pause or interrupt the automation process if unexpected actions occur.
3. Establish a mechanism to track and log the executed steps for audit and accountability purposes.

Risk: Low Data quality and unreliability

Likelihood: Likely

Impact: Major

Risk Mitigation:

1. Regularly update and validate the data sources to ensure accuracy and reliability.
2. Test with multiple different documents to make sure we have achieved varied GPT understanding and covered test suites.
3. Integrate error-checking and validation steps within the automation process to identify potential inaccuracies or inconsistencies.

Risk: Lack of Privacy and Security Concerns

Likelihood: Likely

Impact: Major

Risk Mitigation:

1. Make the majority of it run on the host's computer which is already a person they trust (Mr. Taylor)
2. Implement access controls and authentication mechanisms to ensure that only authorized users can access and interact with the system.

Risk: Ethical implications

Likelihood: Unlikely

Impact: Minor to Moderate

Risk Mitigation:

1. Regularly evaluate and audit the AI system for possible discriminatory actions that we want to minimize.
2. Implement fairness and diversity measures to mitigate biases and stereotypes in decision-making.

Risk: Lack of human intuition and adaptability

Likelihood: Likely

Impact: Moderate

Risk Mitigation:

1. Provide a mechanism for users to intervene or provide additional guidance during the automation process.
2. Use Agile to make sure the client is in the loop during the entire time.

V. Definition of Done

Our definition of done was to have a working, improved version of BabyAGI with LangChain that can take in one of our predefined input questions and give back a good summary at the end that is a culmination of all its tasks using the three agents we defined (PDF Search Agent, Internet Search Agent, and Default GPT agent). We want it to follow this flow:

1. We have an interface to take in a prompt (Console, UI) where the user puts in their question and existing document.
2. After the initial information is given, the program sends a prompt that makes a list of 10 tasks.
3. The script then goes through each task one by one, determining the agent needed and then completing it.
4. The main agents are the Pinecone database access (with the input document), the internet, or default ChatGPT.
5. For the Internet agent, we scrape the results using GoogleAPI, a tool used to get information from websites, and then use GPT to come up with a summary for that one task.
6. For the database, we used a LangChain implementation to reference the document from the Pinecone embeddings and come up with a summary for that one task.
7. Otherwise, we will use the normal ChatGPT response as the summary.
8. Go back to the main task list and continue completing each task, using the correct agent, and getting a summary result until all are finished.

9. At the end of all tasks, show the summary from each task and then make a master summary concluding the report based on the initial question.

Input Question Examples

1. “Here’s a 10k report, from this and the internet, please summarize the key risks that would come from investing in this company?”
2. “Here are the product specs that I want to bring to market, can you do product research to find other possible key competitors that may have a similar product?”
3. “Here are the product specs that I want to bring to market, can you do product research to create a possible business plan that may succeed based on the current landscape for my product?”

Sample output Answer (for 10k)

Sure, looking at the 10k report I’ve identified the steps to find the potential risks

1. Research the current state of the company (Will use Internet Agent)
2. Summarize the research section of 10k Report (Will use Pinecone Agent)
3. Determine the best way to understand these risks in the context of both agents (Will use ChatGPT agent)
4. ...(8-12 total tasks)

So, in summary, the risks of Tesla seem to be the complexities of the supply chain problems, leadership, competition, and regulatory risks.

VI. System Architecture

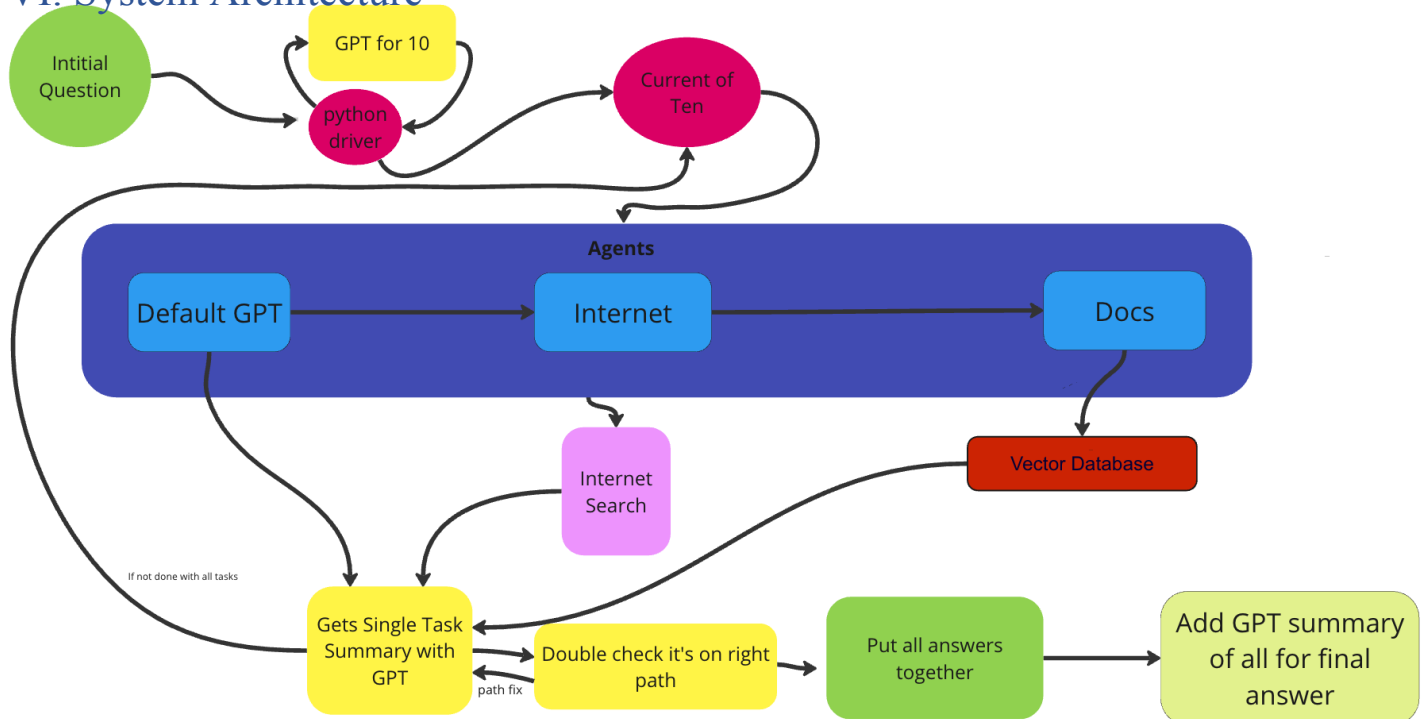


Fig. 1: Basic Overview of Flow for all tasks.

Top-Down Overview

A top-down explanation of how each of the tasks flows together in a more generalized way can be seen in [Fig. 1]. Firstly, the initial question is entered by the user. The question is then sent to the Python driver script which generates 8-12 tasks using the GPT API. These tasks are automatically generated by the default GPT based on user prompts, and they will be assigned to different execution agents to execute. Once that async function has returned, the driver hands off to a loop that goes line by line from the given GPT tasks. For each task in the loop, the current of the ten items chooses which agent to use by asking the GPT which of the agents most closely resembles the given task. From there, the agent either uses the given documents that are accessed in the vector database, the internet agent which scrapes using GoogleAPI, or the default GPT agent which simply asks the GPT API what the answer is. Whichever agent is chosen is eventually defaulted back to the GPT API to make an answer to the given task. Once that answer is provided, it stores them then goes back up to the next of the ten commands. Once all tasks have an answer, the GPT summary is checked against another GPT API call which guarantees that it is still in line with the original task. If not, it goes back up to the current of ten for the given task. Once all tasks have been completed and checked to be on the right path, all the answers are compiled into one main summary which is outputted at the end of all the sub-summaries.

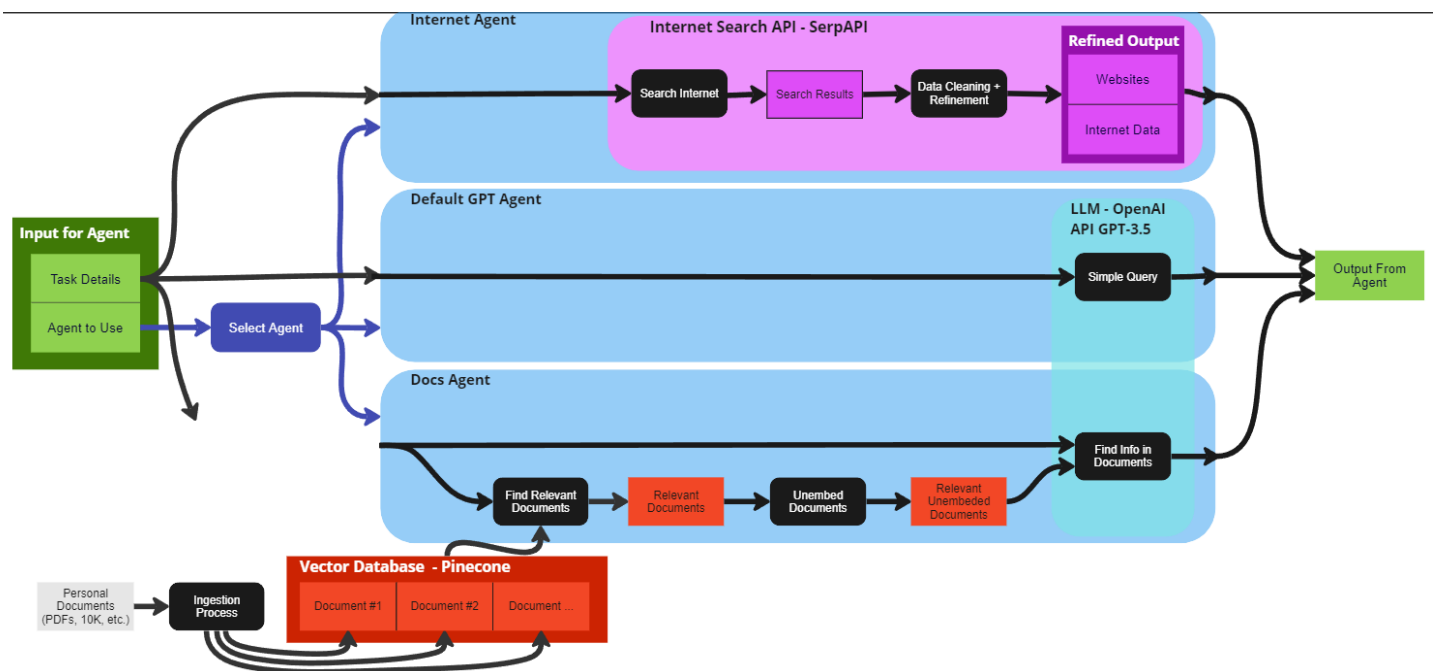


Fig. 2: Flow for an individual task.

After passing in the original question, our script calls the GPT agent which asks for the question asynchronously. From this, the previous history and the next question are combined to go to the LLM which has three main agents. These are briefly explained above, as shown in [Fig. 2], but here will be explained in more detail as the previous example was showing each task in the overall flow of the program. From this context-inclusive prompt, the program is prompted again against itself to determine the proper agent.

If the task requires the internet agent, it will be sent to GoogleAPI and then this will explore a long list of websites that will then be put back to the internet search agent to be sent to data. Additionally, this website ping will be sent to compile a paper trail and the charges will be calculated.

If the task requires the general standalone question agent, it will simply be piped back to GPT to generate an answer.

If the task requires information from the PDF, it must prompt a given namespace from the document which has been through the ingestion process and adheres to certain namespaces. This relevant namespace from the embedded document will then be unembedded to get an actual database result and not simply a piece of a vector database which will produce a relevant document that can be sent to the data retrieved and to the paper trail that must be compiled.

Finally, whether the data retrieved was from the search result agent's output, the normal prompted GPT output, or the LangChain output; one last pass through the LLM will summarize the result and feed that as the final result for the one individual task.

VII. Technical Design

In this section, we will go over how we put each part of these architectural pieces together and more specifically how we tested these ideas together as bringing them into one cohesive structure required a design whiteboard full of diagrams to figure out how to efficiently complete this task.

We started off with the most basic designs for the tests to mesh these pieces starting with each measurable testable integration part. The first major test we had to design for this technical portion was the task creation test.

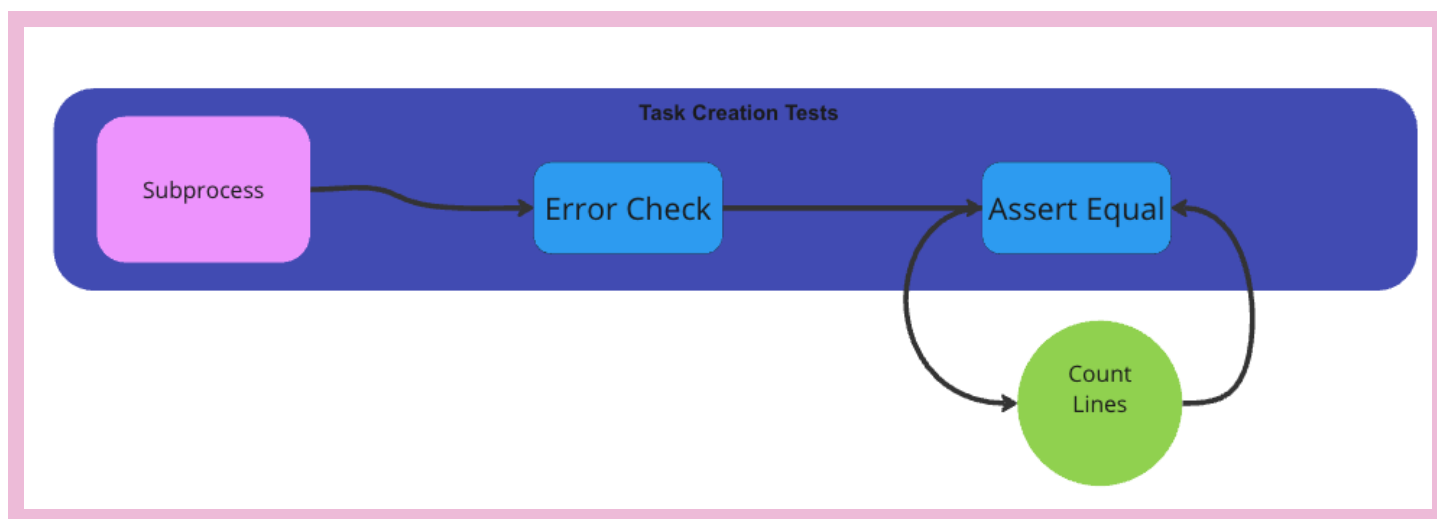


Fig. 3: Technical Design for Task Creation Task

This part shows how we would go about testing if the task creation tests fall into a certain boundary. As with GPT, it would be different every time we run this so we wanted to use assert statements for each of the diagrams to make sure it was robust. As seen in the figure, we would start a subprocess called babyCatAGI.py that would

create the 8-12 tasks but we would do this in the test itself. In this part, we would then error check to make sure we could resolve a variety of issues at this exact step. Specifically, we ran into a lot of string parse issues that were different each time so it was important to have this error check step. Finally, this technical test design included a final assert equal that would count the number of tasks created which we wanted to fall between 8-12 tasks for the times we ran it.

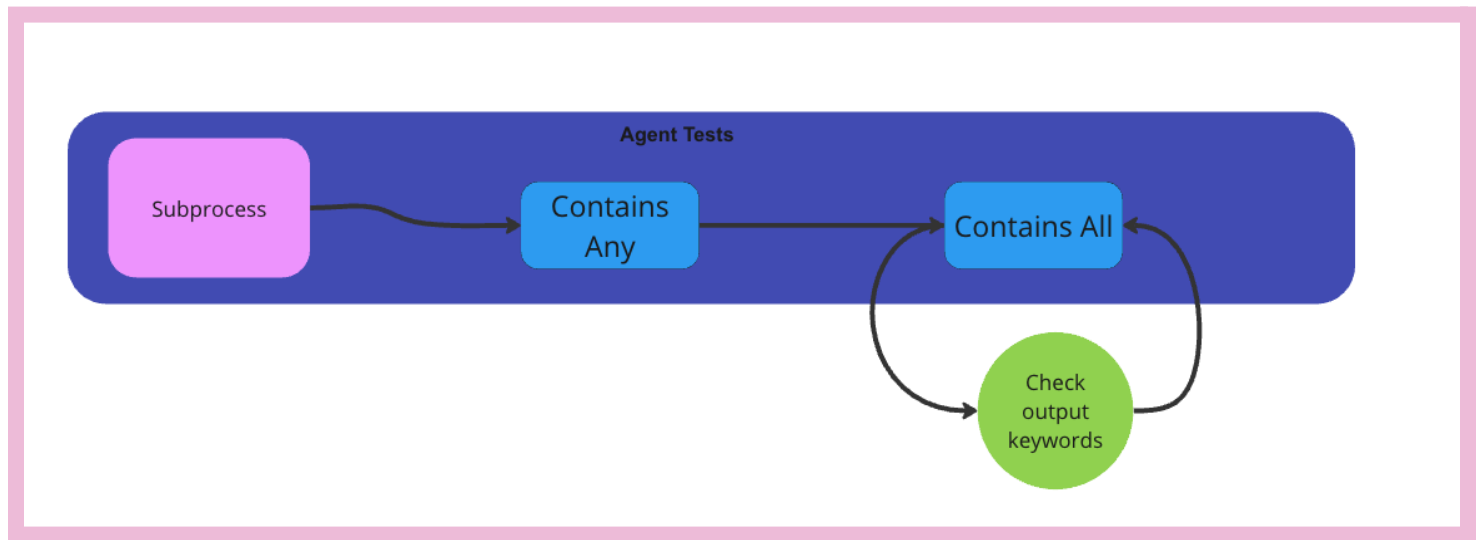


Fig. 4: Technical Design for Agent Tests

This second technical design was for the agent tests. Similarly to the task creation task agent, we focused on a small portion of the code that could be called as its own Python driver. In this case, we ran the same babyCatAGI.py file to test two things. We wanted to make sure it had at least one of each of the agents and that this agent was indeed one that was real. This meant that we needed to design a helper function to check the output keywords to make sure it didn't hallucinate a tool that it didn't have access to.

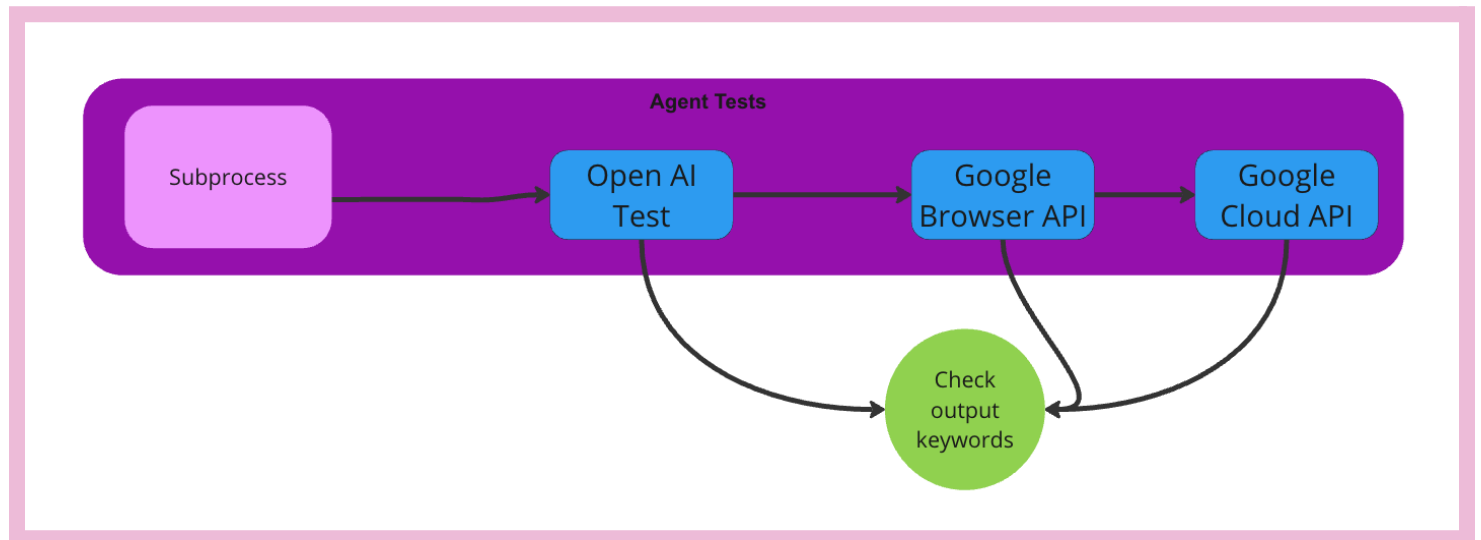


Fig. 5: Technical Design for Agent Tests but for API checks

This final design made sure all the API checks were successful. This was done similarly to all of the other portions of these designs, starting with running a subprocess. From this subprocess, we could determine what the output of the tests was. We then had a helper function check for specific output keywords from these three

API's. Luckily, most of the errors that would be thrown from the API would be constant in how it was worded so we could just parse through these words and make sure that there weren't any errors. By asserting this multiple times, this section could be concluded as working functionally.

Once the technical designs for each of these test portions were fleshed out, it was possible to combine all our diagrams and discrete parts into one master technical design diagram.

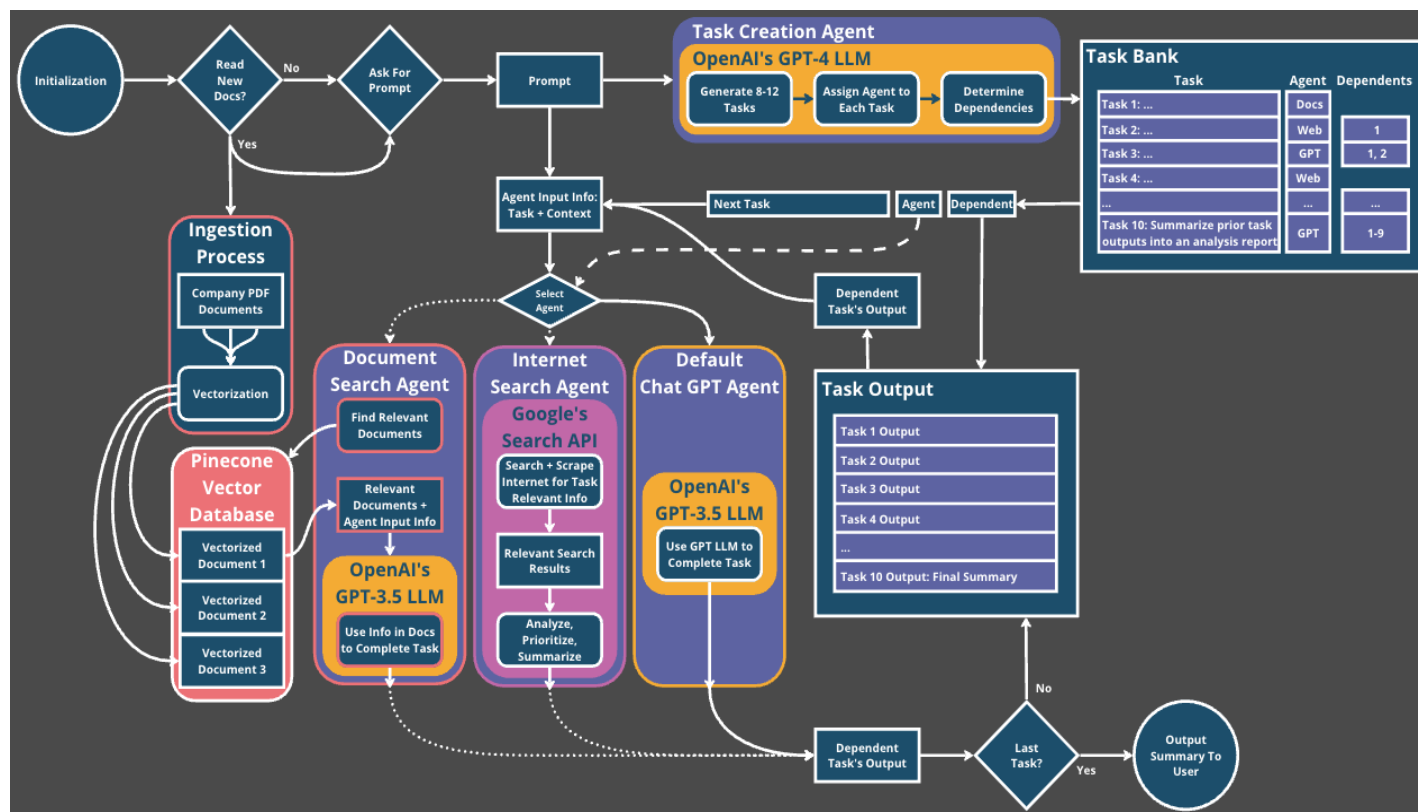


Fig. 6: Master diagram showing each part works together.

First, in the initialization process there is an initial question that asks the user if new documents will be uploaded. If these documents must be read in, it goes to the ingestion process which uses a combination of our previous company PDF documents and vectorizes them in a way that can be efficiently accessed later in the Pinecone vector database. If there are no new documents, it asks for the prompt which is sent to the task creation agent which generates the given 8-12 tasks and assigns dependencies and agents for each task. This falls into a large task bank which lists them out to the user in an understandable way that can be accessed later. From the dependencies and the previous bank, it adds the context and then uses the correct agent. There are three agents implemented that are of particular focus in this instance. The first is the document search agent that finds relevant documents from the vector database and then uses another LLM call to make sure it is complete. The second agent is the internet search agent which uses two Google APIs to find relevant search results and analyze, prioritize, and summarize this data. Finally, the last agent is a simple self-reference GPT Agent that answers the initial prompt with one OpenAI LLM call. These outputs are then put in the task output once it is complete and then update the task output bank outputs the summary to the user.

VIII. Software Test and Quality

Interface Test

(Interface (console or UI))

Purpose:

- To ensure client usability and ease of use, as well as make sure documents are properly inserted.

Tests:

- From the initial mockup of UI design, we had customer(3 people) blind study where we observed without giving them specific directions beforehand.
- For the backend, we tested different API endpoints for post, and got it working with Postman.
- For the front end, we tested every button and interaction individually.

Tools:

- Postman, assert condition python, react, MongoDB.

Edge Cases:

- If the user tester tries something unexpected beyond our UI, other endpoints could be called, the user tester might click on something other than on a button.

Test Result:

- The test passed. The users were able to use it according to the hints given by the system and get answers related to the documentation.

Task Creation Test

(Makes 8-12 goals based on API)

Purpose:

- To make sure that the system is looping enough times to increase the efficiency of output, ensuring that even if a prompt is easy to answer, the model double-checks its own work, OR if a prompt is very complicated, the model won't run for a long time and do multiple API calls.

Tests:

- We will check a couple of iterations of task generation to ensure that the number of tasks generated each time is between 8-12 (usually the number will land around 10, but we don't want any less).
- We make sure to run it 10 times in the assert statement and that there aren't any breaks before an output gets returned.

Tools:

- Assert condition python.

Edge Cases:

- The amount of tasks is beyond 8 to 12, some errors occur that happen not to be within the 10 assert threshold.

Test Result:

- Generally the test could generate 8-12 tasks and pass all of them, but occasionally there were no tasks generated or more than 12 tasks. The probability of such failure was very small. These tasks are automatically generated by GPT, and we could not control the number of tasks precisely at first.

Agents Test

(Determines if those goals are internet (SerpAPI), base agent (Default ChatGPT), or vector task(LangChain from documents stored in Pinecone))

Purpose:

- To make sure that the agents can work to generate tasks.

Tests:

- Run 10 times in the assert test, make sure it doesn't choose an agent that isn't web, docs, or gpt.
- Run each agent individually to see no errors in the 10-time assert test.
- Run a coverage test to see that all get hit in one task creation instance- all agents should be used at least once.

Tools:

- Assert condition python, coverage test python library.

Edge Cases:

- 10-time assert works but error not found, coverage test shows all covered but still doesn't reach assert requirements.

Test Result:

- This test generated tasks that could be executed by any of the three of our existing agents, and passed in multiple runs.

Task Summary Test

(Finishes each goal with a summary/Combines all into a final report with a report for the summary as well)

Purpose:

- To make sure that each goal can be used as a new prompt to complete the goal and to summarize the result of this goal.

Tests:

- Run with_faiss.py (the executable python file that runs the whole project) in 100 assert statements, making sure there is a return value for a summary.
- Gets summary from BabyCatAGI without with_faiss.py in 10 assert statements.

Tools:

- Assert condition in Python.

Edge Cases:

- Some error occurs that happens to not be within the 10 assert threshold.

Test Result:

- Each task summary was generated as the tasks were completed in sequence. And the summary report of the last task answered the prompt we initially entered.

Implementation Display Test

(Be open to the process of the task list, showing each part as it goes so the user knows that it's working)

Purpose:

- To make sure the user can see the full implementation of the system.

Tests:

- Show each individual part either on the UI or terminal.
- Coverage test using Python coverage library. This will show which parts of the code are hit as this is dynamic.

Tools:

- Assert condition python, coverage test python library.

Edge Cases:

- Unexpected output in UI or terminal.

Test Result:

- In multiple runs, our output was able to be displayed in the terminal in the format we desired.

OpenAI API test

(Add safety measures to make sure the openAI API isn't throttled too far)

Purpose:

- Make sure that OpenAI doesn't throw a rate error that stops the program's execution.

Tests:

- Assert 10 times the running of BabyCatAGI, the current version of BabyAGI we are using, looking for openAI exceptions.

Tools:

- Assert condition python.

Edge Cases:

- Some error occurs that happens to not be within the 10 assert threshold.

Test Result:

- No errors were thrown after all 10 runs.

Search Sources Test

(Test for Pinecone and Internet)

Purpose:

- Make sure that there is a Pinecone database accessible and the Google API calls will work.

Tests:

- Used previously created Pinecone test to check that requests and insertions can be made given the keys.
- Used pytest library to check for API call usability before running in the assert statement.

Tools:

- PineconeTester.py, pytest, Assert condition python.

Edge Cases:

- Pinecone instance error with the free account, API errors that are not accounted for in pytest.

Test Result:

- By running the Pinecone test file several times, it became clear that our document data was stored in the Pinecone database and could be used as a resource to answer our questions. We ran the search API several times, and had lots of good website findings that were very diverse.

*All nonfunctional requirements will be met through how we implemented all these tests. We combined both as we were making them up.

IX. Project Ethical Considerations

When developing software that utilizes APIs to connect to ChatGPT and the internet for task completion, it is crucial to carefully consider and address ethical considerations. Here are some key aspects to keep in mind:

ACM: 1.2, 1.4, 2.6

Specifically, these were “avoiding harm”, “being fair to not discriminate”, and “only working in areas of competence”.

IEEE: 1, 2, 7, 8

“Human Rights”, “Well Being and Data”, “Data Agency”, “Accountability”

To address these principles, we will

1. **Privacy and Data Security:** Ensure that user data and information are handled securely. Implement robust encryption measures, adopt privacy-focused practices, and follow relevant data protection regulations. Minimize the collection and storage of personally identifiable information unless necessary for the functioning of the software.
2. **Informed Consent:** Obtain explicit consent from users before accessing or utilizing their personal data. Clearly communicate the purpose and scope of data usage and adhere to the principles of transparency and user control.
3. **Bias and Fairness:** Be mindful of the potential biases inherent in AI models and data sources. Take proactive steps to mitigate bias during model training and ensure that the software doesn't perpetuate or amplify existing biases, discrimination, or unfair treatment. Regularly monitor and evaluate the system's outputs to identify and address any unintended biases.
4. **Accountability and Liability:** Clearly define the responsibilities and limitations of the software. Communicate any potential risks or limitations to users. Establish mechanisms for addressing errors, providing support, and handling user complaints or concerns. Take responsibility for the actions and consequences of the software.
5. **User Safety:** Prioritize the safety and well-being of users. Consider potential risks associated with executing tasks, especially those involving physical or financial aspects. Implement safety measures and safeguards to protect users from harm. Clearly communicate any limitations or potential risks to users.

6. Transparency and Explainability: Strive to make the decision-making process of the software transparent and understandable to users. Provide explanations for how tasks are assigned, which agents are used, and how the system arrives at its final summary. Enable users to access and understand the underlying processes and algorithms.

7. Legal and Regulatory Compliance: Ensure compliance with relevant laws, regulations, and industry standards. Stay informed about emerging guidelines and best practices related to AI and chatbot systems. Adapt the software accordingly to meet evolving legal and ethical requirements.

X. Results

We started this project based on the BabyCatAGI open-source repository, with our main Python script being “with_faiss.py”. When we started working on it, it was already functional, but there were multiple features and improvements that we hoped to add.

The first main hurdle was making sure we could integrate all the libraries with minimal errors. This was an amalgamation of 2 GitHub repositories and led to multiple unit tests along the way to make sure everything worked from start to finish. This encompassed: API Tests, Task Tests, and Integration Tests.

This led us down a route to make sure that this was fine-tuned. We thought the best way to do this was to break up the process into a couple of categories: Good data, Good prompts, Temperature, New Tools, Self Referencing Along the Way.

The first of these was simply tweaking the factors that would alter ChatGPTs response including the prompts and temperature. The temperature alters the randomness of ChatGPTs responses, and after testing the product with different temperature levels, we decided that a low temperature of 0.1 or lower was optimal since we are answering business queries and are prioritizing accuracy. As for the prompts, we tested a few slight modifications to the wording to see if it would change the quality of the responses. One of these modifications was asking GPT to act as a professional in the relevant field so that it did not output responses with filler words or seem unsure of itself. This addition did make a slight difference, though it could be beneficial to continue trying new prompts to see if any result in better outputs. After we got some good outputs, we showed them to a small group of people that gave us user feedback; including things like showing the process, try not to let it follow dead ends, show the final result with tables or visual aids, make a plan to address risks rather than just pointing them out, make objective more specific, and have a more professional output. Using some of this feedback, we modified the prompts and code to better fit these standards.

The second part that we focused on was adding features that we thought would be useful. Through research, we found that if you ask GPT to review the accuracy of its initial response, it is likely to find any mistakes it may have made which dramatically improves the accuracy. We also created a new agent that is more directly related to the needs of our product. This agent was the Docs agent, which uses LangChain to parse and use information from the PDFs. The final, and arguably most important feature that we implemented was the UI. The UI will make our product more accessible, which is important since one of the main purposes of Analytical Data Systems is helping companies implement LLMs and Artificial Intelligence into their workflow. The UI allows for members of said companies to more easily use our product directly. Finally, we used one last GPT “check” at the end to make a summary bringing each part together, incrementally making a final picture.

As a result of our testing so far, our system can generate 8-12 tasks based on user-entered prompts, and it is able to execute tasks to generate outputs using the documents provided by the user, the internet and the default GPT. The accuracy of the final summary report will be related to the complexity of the input prompts, so we will put the user feedback on the report back into the prompt to train the system to get a better answer. Our system can also connect to both the OpenAI API and the Google Search API, separately from task completion. Our client wanted us to use an additional vector database that was different from Pinecone, called Snowflake. This addition to our requirements was during the fourth week of working on the project. And although tests were indeed created, as this part wasn't directly needed, we didn't go further with work on Snowflake. And while we didn't manage to implement it as a part of our main system, we did have a test for Snowflake to show that, given more time, Snowflake could absolutely be used with our software. This test was broken into five subtests that tested Snowflake connection, setup, insertion, query, and deletion. Each sub-test tested a different process or command that would be vital to the future implementation of Snowflake into our project.

As for features that we planned on implementing but were unable to due to time restrictions, one related to customization which would allow the user to choose whether the script uses GPT-3.5-Turbo or GPT-4 when the script runs. This was operating properly at first, but then it was creating a bug with outputs that were in random characters and Korean. If this bug were fixed, this could be implemented into the UI and be a useful feature. The reason this is important is because GPT-4 is more accurate in its responses but costs more per call and is slower, so depending on what is important to the customer, they could choose what to prioritize. Also, we were going to integrate a feedback system on a company-by-company basis in which you would have the customer review the outputs, then feed those reviews back into ChatGPT to get responses catered to that customer's needs.

XI. Future Work

In order to continue where we left off, one would need to follow the setup process detailed in our readme file for both Windows and Mac, which is fairly straightforward. There are no specific system requirements for our code because it does not put as much strain on the computer as it does on ChatGPT and the other APIs themselves. Near the end of the allotted time we were given to work on the project, our client talked to us about implementing a secondary vector database called Snowflake that would work with our code as well as with Pinecone. While we didn't end up having enough time to implement Snowflake, a Snowflake test file is in our code showing how one could easily integrate it with our project. Another piece of code that can be added in by future development teams would be to ask the user if they want to re-generate the initial list of tasks.

Additionally, there are a couple of promising methods that we felt could have been very useful to the fine-tuning of this algorithm that didn't really prove to be useful in this five week's time. Specifically, and most importantly, we would now focus on "parameter efficient fine-tuning" as Google has now released a very useful cloud tool where this could be done. Parameters seemed like too much to deal with as these LLM's are by definition, large. But some further focus on this area would be very important in this future fine-tuning process.

Moreover, there is a lot more to be done to make sure this project is accessible to others. One idea we toyed around with routinely was the implementation of a plugin. This would be a great way to directly interface our code with the user via the OpenAI website itself. This user accessibility problem was partially done through the

use of the website but a plugin function could have still added a lot. There were problems with this during the actual coding we were doing such as the plugin store being locked down so even if we wanted to go forward with this plan, it wouldn't have been very practical. As the end of the progress came to light, it became obvious that there would be more and more plugins enabled on the store so this would be a more viable option in the future.

Finally, and possibly one of the most important additions as time goes on, is the ability for this project to have a more scientific way to judge people rating our prompts. As we rounded the corner and finished the project, we realized this was one of the largest points that needed more info. Although we did get a lot of feedback, simply giving GPT the feedback led to multiple control issues. In the future, having a good way to distill data into very basic and understandable instructions would be of use. We would implement similar tables and dedicated GPT snippets as we did before but add to it additional questions that get more specific answers from the interview. Additionally, we would write more documentation on how to turn these interview notes into actual actionable notes that we could add to the final prompt to make it better. This would be less programming heavy but nevertheless, important for fine-tuning in the future.

XII. Lessons Learned

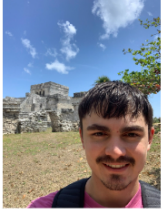
When we began this project, none of us had much experience with AI and LLMs, so our first course of action was to talk to our client about what he wanted and do lots of research. As for knowledge, we used multiple coding languages including Python for the functional portions, and JavaScript for most of the UI. As long as a future developer has a general understanding of computer science and has read through the report, with some research they should be able to continue where we left off. Depending on how much a future developer plans to work on it, the things we could have improved on slightly was sharing the research that we found with each other more effectively. Throughout the project, we would all come together and work as needed, but there were also times when we would be working on portions on our own, then would have to spend time catching our teammates up on what we did. This comes down to communication, and with slightly better communication we could have been more efficient. The second portion of the product was a mix between implementation and modifications to our plan. Throughout the whole process, we were brainstorming with and without our client on how we can make a proficient final product so we were constantly coming up with new ideas and modifying our plan. It could have been better to spend more time setting concrete goals then reaching those goals before moving on to the next, but this was also due in part to the fact that our client wanted us to act as more of a research team so that he had the necessary knowledge to implement this product in the future. Also, we documented most of what we did but not perfectly, and it could have been beneficial to spend more time writing down what we were doing and learning. Overall, the project went well and there's not much we would have changed about it. We all put in effort to make a final product we would be happy with, and the outcome was satisfactory.

XIII. Acknowledgments

We would like to thank our client, James Taylor, for introducing us to such an interesting and innovative topic. This field session not only gave us the experience needed to function in a workplace but also a deeper understanding in the field of Artificial Intelligence. This project always kept us on our toes and ready to start on

the next section. We would also like to thank our advisor, Zibo Wang, for guiding us through the process of bringing our product to fruition. He was a huge help whenever we had an issue and was always available to provide assistance if we ever needed it. These two were great to work with and we couldn't have asked for better leaders.

XIV. Team Profile



- Aaron Matthew Jones II
- Senior
- Major: Computer Science with a Focus in AI and Cyber Security
- Hometown: Antonito, CO
- Work Experience: Ranch Hand at Rancho Del Oso Pardo and Rancho Lobo Ltd.
- *I am interested in where the future of this product lies and am very excited to work on it myself.*



- Jason Lee
- Senior
- Major: Computer Science General Focus Minor in Public Affairs. Working on a Master's Degree.
- Hometown: Colorado Springs, Colorado
- Work Experience: Research funded by Meta (VR 3d Printer), Google (Deep learning embedded devices security), internship at Eclypses
- *Interested in the application of intelligent machines to uplift deeply personal human goals.*



- Yonghao Li
- Senior
- Major: Computer Science with a Focus in Computer Engineering
- Hometown: Fijian, China
- *I am interested in all things in the computer science field and how they can be applied in real life.*



- Andrew Vargish
- Junior
- Major: Computer Science with a Focus in Data Science
- Hometown: Castle Rock, CO
- *I am interested in the future of artificial intelligence and how it will reshape how we use the internet*

References

- [1] “Langchain: Introduction and getting started,” Pinecone, <https://www.pinecone.io/learn/langchain-intro/> (accessed May 19, 2023).
- [2] “GPT-4 tutorial: How to chat with multiple PDF files (~1000 pages of Tesla’s 10-K annual reports),” YouTube, <https://www.youtube.com/watch?v=Ix9WIZpArm0> (accessed May 19, 2023).
- [3] yoheinakajima. “Yoheinakajima/Babyagi.” GitHub, 12 May 2023, github.com/yoheinakajima/babyagi. Accessed 19 May 2023.
- [4] DarylRodrigo. “DarylRodrigo/Auto-Gpt-Ts: Implementation of Auto-GPT in Typescript.” GitHub, May 2023, github.com/DarylRodrigo/auto-gpt-ts. Accessed 19 May 2023.
- [5] shamspias. “Shamspias/Langchain-Chat: Langchain-Chat Is an AI-Driven Q&a System That Leverages OpenAI’s GPT-4 Model and FAISS for Efficient Document Indexing. It Loads and Splits Documents from Websites or PDFs, Remembers Conversations, and Provides Accurate, Context-Aware Answers Based on the Indexed Data. Easy to Set up and Extend.” GitHub, 19 May 2023, github.com/shamspias/langchain-chat. Accessed 19 May 2023.
- [6] BabyAGI-UI By Container: Vercel.app Year: 2023 URL: <https://babyagi-ui.vercel.app/>
- [7](*HOW to WRITE a BUSINESS PLAN Managing and Planning Series*) URL: <https://www.sba.gov/sites/default/files/How%20to%20Write%20a%20Business%20Plan.pdf>

Appendix A – Key Terms

Term	Definition
<i>AGI</i>	AGI stands for Artificial General Intelligence. This term refers to a type of artificial intelligence that has the capacity to understand, learn, and apply its knowledge to a broad range of tasks at a level equal to or beyond human capability. Unlike narrow AI, which is designed to perform a specific task, such as image recognition or language translation, AGI can theoretically perform any intellectual task that a human being can do.
<i>API</i>	API stands for Application Programming Interface. It is a set of rules and protocols for how software components should interact. APIs define the methods and data formats that a program can use to communicate with other software or components, such as libraries, operating systems, or even other services over the internet. They simplify programming by abstracting the underlying implementation and only exposing objects or actions the developer needs. APIs are used in all kinds of programming from web development to operating systems to machine learning applications.
<i>BabyAGI</i>	BabyAGI is an innovative artificial intelligence (AI) platform designed to train and evaluate different AI agents within a simulated environment. It is a simplified version of the original Task-Driven Autonomous Agent developed and launched by AI expert Yohei Nakajima.
<i>BabyCatAGI</i>	BabyCatAGI, which stands for Baby Cognitive Architecture for Artificial General Intelligence, is a highly advanced technology that aims to make machines as intelligent as humans. It utilizes sophisticated algorithms and machine learning techniques to process a vast amount of information, enabling it to make intelligent decisions. BabyCatAGI is designed to be adaptable, capable of handling large amounts of data, learning from it, and improving its performance on tasks over time. It is able to solve a wide range of problems and even comprehend and use human language.
<i>GPT</i>	GPT stands for "Generative Pre-trained Transformer". It's a type of artificial intelligence model developed by OpenAI for natural language processing tasks. GPT is designed to generate human-like text by predicting the probability of a word given the previous words used in the text. It's pretrained on a large corpus of text from the internet and then fine-tuned for specific tasks, like translation, summarization, or question answering.
<i>LangChain</i>	A framework for developing applications powered by language models

<i>LLM</i>	LLM stands for Large Language Model. It refers to machine learning models that are trained on a vast amount of text data and are capable of generating human-like text. These models, like GPT-4 developed by OpenAI, are designed to understand context, make predictions, and produce responses based on the information they have been trained on. They're used in a variety of applications, including text generation, translation, summarization, and more.
<i>Pinecone</i>	A vector database that facilitates the creation of high-performance vector search applications. It is designed to be developer-friendly, fully managed, and easily scalable without the need for managing infrastructure.
<i>10-K</i>	A 10-K report is an annual report required by the U.S. Securities and Exchange Commission (SEC) that gives a comprehensive summary of a company's financial performance.